

XAMS Slow Control — EPICS Design Alternative

Version 0.1 — 16 September 2026

Companion to `DESIGN.md`, which specifies a Python service stack. This document specifies the same system built on **EPICS**, using the hardware inventory and channel configuration recovered in September 2026.

> **Status: considered and set aside — 16 September 2026.** > The decision is for the Python design in `DESIGN.md`. The deciding factor was the CompactDAQ: it carries 20 of the 30 connected channels, it has no maintained EPICS device support, and it therefore requires a Python soft IOC in either design. Choosing EPICS would mean maintaining two paradigms — an EPICS installation *and* a Python IOC — for one small system, rather than one. > > This document is kept because the analysis is sound and the conditions for revisiting it are recorded in §16. It is not a rejected proposal; it is the road not taken, written down.

It is written as a genuine alternative, not as a foil. Where EPICS is better than the Python design, that is stated; where it is worse for this particular system, that is stated too. The decision document (`XAMS-slow-control-options.pdf`) recommends the Python stack; this document is what the other choice actually looks like, so that the recommendation can be judged rather than taken on trust.

1. What EPICS is, in one page

EPICS is the control system used at accelerators, light sources and large detector experiments. Two ideas carry it.

The IOC (Input/Output Controller) is a process that talks to hardware and serves named channels on the network. Historically these ran on VME crates; today an IOC is an ordinary process, and several can run on one PC.

The record is the unit of data. A record is a channel with a type and a set of fields: where the value comes from, how often to read it, its engineering unit, its alarm limits. Scanning, alarming and unit handling are *declared*, not coded:

```
record(ai, "XAMS:PRES:MAIN") {
    field(DESC, "Detector pressure")
    field(DTYP, "asynFloat64")
    field(INP, "@asyn(CDAQ,5)")
    field(SCAN, "10 second")
    field(EGU, "bar")
    field(PREC, "3")
    field(HIGH, "1.8")    field(HSV, "MINOR")
    field(HIHI, "2.0")   field(HHSV, "MAJOR")
}
```

That is a complete, self-describing, periodically scanned, alarming channel. No loop was written, no threshold comparison coded, no unit string carried around by hand.

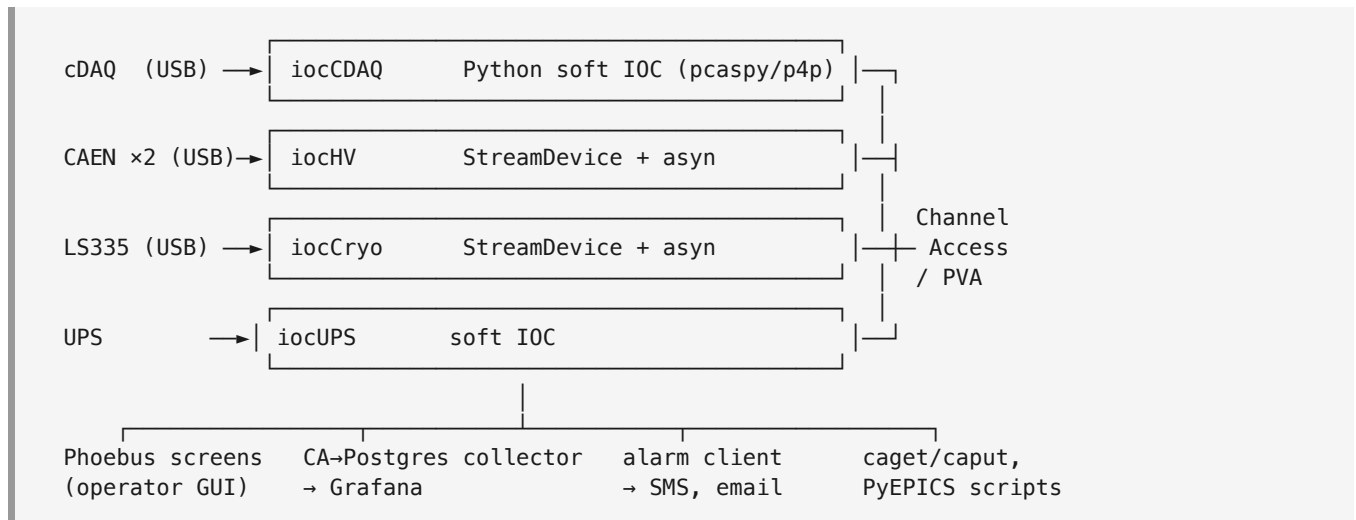
Records are served as **PVs** (process variables) over Channel Access or PVA. Any client on the network reads one by name, with no address and no port:

```
caget  XAMS:PRES:MAIN
caput  XAMS:HV:ANODE:VSET 3500
camonitor XAMS:TEMP:TT201
```

Everything else in EPICS — archivers, operator screens, alarm handlers — is a client of that namespace.

2. Architecture for XAMS

Four IOCs, one per device family. Each is an independent process that can be restarted alone: a crashed vacuum IOC does not take HV monitoring down.



The split is forced by the hardware. The two serial instruments map cleanly onto StreamDevice, which is EPICS at its best. The cDAQ does not: there is no maintained EPICS driver for NI-DAQmx, so that IOC is written in Python regardless. This is the single most important fact in this document and it is discussed in §5.

3. PV naming

EPICS names are a flat global namespace, conventionally `<system>:<subsystem>:<signal>`, uppercase, colon-separated. The tags recovered from the LabVIEW system map directly:

Tag	PV
P101 ... P104	XAMS:PRES:P101 ... XAMS:PRES:P104
Pmain	XAMS:PRES:MAIN
FM101	XAMS:FLOW:FM101
TT201 ... TT207	XAMS:TEMP:TT201 ... XAMS:TEMP:TT207
TT301 ... TT304	XAMS:TEMP:TT301 ... XAMS:TEMP:TT304
TT103 , TT104	XAMS:TEMP:TT103 , XAMS:TEMP:TT104
TTAMB	XAMS:TEMP:AMB
HV channels	XAMS:HV:ANODE:VMON , :VSET , :IMON , :STAT , ...
Lake Shore	XAMS:CRYO:TA , XAMS:CRYO:SETP , XAMS:CRYO:HTR

Tag	PV
UPS	XAMS:UPS:ONBATT , XAMS:UPS:CHARGE

Unlike MQTT topics, this namespace is not a local convention: any EPICS tool in the world understands it, and `caget XAMS:PRES:MAIN` works from any machine on the network without configuration. That is a real advantage, and also a security consideration (§11).

4. Records — the cDAQ channels

Generated from a substitutions file rather than written out by hand. The template:

```
# db/rtd.template
record(ai, "XAMS:TEMP:$(TAG)") {
    field(DESC, "$(DESC)")
    field(DTYP, "asynFloat64")
    field(INP, "@asyn$(PORT),$(CHAN)")
    field(SCAN, "10 second")
    field(EGU, "C")
    field(PREC, "2")
    field(LOW, "$(LOW=-200)")    field(LSV, "MINOR")
    field(HIGH, "$(HIGH=100)")   field(HSV, "MINOR")
    field(SIML, "XAMS:SIM")      field(SIOL, "XAMS:SIM:$(TAG)")
}
```

and the instantiation, which is the whole RTD map in one file:

```
# db/xams.substitutions
file "rtd.template" {
    pattern { TAG,    PORT,    CHAN, DESC }
    { TT201, CDAQ_9226, 0, "TBD - location" }
    { TT202, CDAQ_9226, 1, "TBD - location" }
    { TT203, CDAQ_9226, 2, "TBD - location" }
    { TT204, CDAQ_9226, 3, "TBD - location" }
    { TT205, CDAQ_9226, 4, "TBD - location" }
    { TT206, CDAQ_9226, 5, "TBD - location" }
    { TT207, CDAQ_9226, 6, "TBD - location" }
    { TT301, CDAQ_9216_1, 0, "TBD - location" }
    { TT302, CDAQ_9216_1, 1, "TBD - location" }
    { TT103, CDAQ_9216_1, 2, "TBD - location" }
    { TT104, CDAQ_9216_1, 3, "TBD - location" }
    { AMB, CDAQ_9216_1, 4, "Ambient" }
    { TT303, CDAQ_9216_1, 5, "TBD - location" }
    { TT304, CDAQ_9216_1, 6, "TBD - location" }
}
```

Fourteen RTDs; `9216_2` is entirely unconnected and gets no records.

The 9207 voltage channels carry the recovered scaling. EPICS applies it declaratively through `ASL0 / A0FF`, so no arithmetic is written:

```

record(ai, "XAMS:PRES:P101") {
    field(DTYP, "asynFloat64")
    field(INP, "@asyn(CDAQ_9207,0)")
    field(SCAN, "10 second")
    field(AOFF, "-1.0")           # offset subtracted, per the LabVIEW panel
    field(ASLO, "25.0")          # multiplier
    field(EGU, "TBD")            # units not recovered – see DESIGN.md §4.2
    field(SM00, "0.0")
}

```

PV	Channel	AOFF	ASLO	EGU
XAMS:PRES:P101	9207/ai0	–1.0	25.0	TBD
XAMS:PRES:P102	9207/ai1	0	1.0	TBD
XAMS:PRES:P103	9207/ai2	0	1.0	TBD
XAMS:PRES:P104	9207/ai3	0	1.0	TBD
XAMS:PRES:MAIN	9207/ai5	0	0.714	TBD
XAMS:FLOW:FM101	9207/ai7	0	6.0	g/min

9207/ai4 and ai6 (v4, v6) carry generic names and are probably unused; ai8:15 likewise. They get no records until confirmed.

Note the flow unit. The LabVIEW front panel labels FM101 as "Flow (SLPM)". That label is wrong — it is a mass flow in g/min. Recorded here so the error is not reintroduced from the old panel.

5. The cDAQ IOC — where EPICS does not help

There is no maintained EPICS device support for NI-DAQmx. Two options:

(a) Write an asyn port driver in C++ against the nidaqmx C API. Real work, and permanent maintenance: it must be rebuilt against each EPICS base and each DAQmx release, by someone who knows both C++ and the asyn driver model.

(b) A Python soft IOC using pcaspy (Channel Access) or p4p (PVA), wrapping the same nidaqmx package the Python design uses.

Option (b) is the only sensible choice, and it means **the cDAQ code is Python in both designs**. The 20 connected channels — the backbone of the system — are served by a Python process either way. What EPICS adds on top of that is the record layer for the other instruments and the standard protocol.

```

from pcamspy import SimpleServer, Driver
import nidaqmx

prefix = 'XAMS:'
pvdb = {
    'TEMP:TT201': {'prec': 2, 'unit': 'C', 'scan': 10},
    'PRES:MAIN': {'prec': 3, 'unit': 'TBD', 'scan': 10},
    # ...
}

class CDAQDriver(Driver):
    def __init__(self):
        super().__init__()
        self.tasks = build_tasks()      # one per module, on-demand reads

    def read(self, reason):
        return self.tasks[reason].read()

```

The task structure is identical to [DESIGN.md §7.1](#): one on-demand task per module, 1 Hz reads, the RTD configuration handled by DAQmx (PT_3851 / PT_3750, three-wire, r_0 100 Ω / 1000 Ω).

6. The HV IOC — where EPICS is genuinely better

The CAEN ASCII protocol maps onto StreamDevice almost line for line. This is EPICS at its strongest: the device protocol becomes a declarative file, not code.

```

# protocol/caen.proto
Terminator = CR LF;
ReplyTimeout = 1000;

getVmon {
    out "\\$BD:$(\\$1)d,CMD:MON,PAR:VMON,CH:\\$2";
    in  "#BD:%*d,CMD:OK,VAL:%f";
}

getImon {
    out "\\$BD:$(\\$1)d,CMD:MON,PAR:IMON,CH:\\$2";
    in  "#BD:%*d,CMD:OK,VAL:%f";
}

setVset {
    out "\\$BD:$(\\$1)d,CMD:SET,PAR:VSET,CH:\\$2,VAL:%.1f";
    in  "#BD:%*d,CMD:OK";
}

getSerial {
    out "\\$BD:$(\\$1)d,CMD:MON,PAR:BDSNUM";
    in  "#BD:%*d,CMD:OK,VAL:%s";
}

```

and the records:

```

record(ai, "XAMS:HV:$(NAME):VMON") {
    field(DTYP, "stream")
    field(INP, "@caen.proto getVmon($(ADDR),$(CH)) $(PORT)")
    field(SCAN, "2 second")
    field(EGU, "V")
    field(PREC, "1")
}
record(ao, "XAMS:HV:$(NAME):VSET") {
    field(DTYP, "stream")
    field(OUT, "@caen.proto setVset($(ADDR),$(CH)) $(PORT)")
    field(EGU, "V")
    field(DRVH, "${VMAX}")          # hard limit, enforced by the record
    field(DRVL, "0")
}

```

`DRVH` / `DRVL` are worth noting: the record itself refuses out-of-range writes. In the Python design that check is code you write and test; here it is a field.

Channel instantiation, from the recovered names:

```

file "hvchan.template" {
    pattern { NAME,    PORT,  ADDR, CH, VMAX }
        { PMT_TOP, HV1,    0,    0, 2000 }
        { PMT_BOT, HV1,    0,    1, 2000 }
        { TS,      HV1,    0,    2, 8000 }
        { BS,      HV1,    0,    3, 8000 }
        { CATHODE, HV2,    1,    0, 8000 }
        { GATE,    HV2,    1,    1, 8000 }
        { ANODE,   HV2,    1,    3, 8000 }    # CAEN 2, channel 3 - confirmed
        { NAI,     HV2,    1,    2, 2000 }
}

```

TBD: only the anode's supply and channel index are confirmed. The remaining seven assignments above are placeholders and must be verified before any write is enabled.

Serial port setup in `st.cmd`:

```

drvAsynSerialPortConfigure("HV1", "COM8", 0, 0, 0)
asynSetOption("HV1", 0, "baud", "9600")
asynSetOption("HV1", 0, "bits", "8")
asynSetOption("HV1", 0, "parity", "none")
asynSetOption("HV1", 0, "stop", "1")

```

This is where EPICS is weaker than the Python design, however. `COM8` is hardcoded. EPICS has no equivalent of resolving the port by USB hardware ID and then confirming the instrument's own serial number before permitting a write. The identity check of `DESIGN.md §6.2` — the protection against the two identical supplies exchanging COM numbers — has to be built: a `getSerial` record read at startup, compared against an expected value by a sequencer program or a startup script, gating the `VSET` records. Possible, but it is custom work in an environment where custom work is harder.

7. The cryostat IOC — Lake Shore 335

Also `StreamDevice`, and straightforward:

```
# protocol/ls335.proto
Terminator = CR LF;

getTemp { out "KRDG? \ $1"; in "%f"; }
getSetp { out "SETP? 1"; in "%f"; }
setSetp { out "SETP 1,%.2f"; }
getHtr { out "HTR? 1"; in "%f"; }
getRange { out "RANGE? 1"; in "%d"; }
getId { out "*IDN?"; in "%s"; }
```

```
record(ai, "XAMS:CRY0:TA") {
    field(DTYP, "stream")
    field(INP, "@ls335.proto getTemp(A) CRY0")
    field(SCAN, "10 second")
    field(EGU, "K")
}
record(ao, "XAMS:CRY0:SETP") {
    field(DTYP, "stream")
    field(OUT, "@ls335.proto setSetp CRY0")
    field(EGU, "K")
    field(DRVH, "300") field(DRVL, "0")
}
```

PID settings ($P = 100$, $I = 20$, $D = 0$) are read back and displayed; the instrument's own limits remain the authority, exactly as in the Python design.

8. The flow integrator

The Python design makes this a stateful service (`DESIGN.md §7.5`). EPICS does it with a `calc` record and the **autosave** module:

```
record(calc, "XAMS:FLOW:TOTAL") {
    field(DESC, "Integrated mass flow")
    field(INPA, "XAMS:FLOW:FM101 CP") # g/min
    field(INPB, "XAMS:FLOW:TOTAL.VAL NPP") # own previous value
    field(INPC, "10") # dt, seconds
    field(CALC, "B + A*C/60") # -> grams
    field(SCAN, "10 second")
    field(EGU, "g")
    field(PREC, "1")
    info(autosaveFields, "VAL") # survives a restart
}
record(bo, "XAMS:FLOW:TOTAL:RESET") {
    field(DESC, "Reset integrator")
    field(OUT, "XAMS:FLOW:TOTAL.VAL PP")
    field(VAL, "0")
}
```

`info(autosaveFields, "VAL")` is a genuine advantage: persistence across restarts is declared in one line, where the Python design writes the accumulator to disk and reads it back explicitly.

Two things EPICS does **not** give for free here, and both matter:

- **Gap accounting.** Autosave restores the total, but nothing records that the IOC was down for two hours and that the total is therefore an underestimate. The Python design publishes `gaps_s` alongside the value. Reproducing that in EPICS means additional records tracking downtime.

- **Reset as period closure.** The `bo` record above simply zeroes the accumulator, destroying the previous total. The Python design closes a period and starts a new one, keeping the history of how much passed through during each. Doing that in EPICS means an archiving step around the reset.

Note also `C = 10`, the interval in seconds, divided by 60 for g/min. The same factor-60 trap as in the Python design, and here it is a constant in a `CALC` expression where no test covers it.

9. Alarms

What EPICS gives for free. Every record carries `LOLO / LOW / HIGH / HIHI` with a severity per threshold, plus `HYST` for deadband. Thresholds are declared alongside the channel, applied by the record, and visible through `caget XAMS:PRES:MAIN.HIHI`. This is better than the Python design, where the same logic is written and tested.

Crucially, this removes the threshold-duplication problem of `DESIGN.md §11`: because the limits are fields of the record, Phoebus screens and archived data see the same numbers by construction. Nothing has to be generated or kept in step.

What must still be built. Getting an alarm to a telephone:

- **Phoebus alarm server** requires **Kafka**. For one rack that is disproportionate infrastructure.
- **ALH**, the legacy alarm handler, is simpler, but dated and awkward to configure.
- **A Python client** using `pyepics` or `p4p`, subscribing to the alarm severity of the relevant PVs and calling the existing SMS script. Roughly the same code as `alarms/notify.py` in the Python design.

The third is the realistic option — which means the notification layer is Python in both designs, just as the cDAQ layer is.

Staleness is handled better than in the Python design: a record whose read fails goes into `INVALID` severity automatically, with no code. That is a real benefit; a frozen plausible value is the failure mode that matters most.

10. Archiving and plots

Two routes.

Archiver Appliance is the standard EPICS historian: Java, Tomcat, MySQL, and its own storage tiers. It is powerful, well suited to thousands of PVs, and disproportionate for thirty. Phoebus integrates with it directly.

A CA→PostgreSQL collector — a small Python client subscribing to the PVs and writing to the same schema as `DESIGN.md §9.2`, so Grafana works exactly as in the Python design. Perhaps 100 lines, and it keeps the JSONL/Parquet archive and the whole storage design unchanged.

The second is recommended for XAMS. It also keeps the "database is a cache, files are the truth" property, which Archiver Appliance does not naturally give.

11. Operator screens and access

Phoebus (the CS-Studio successor) is a Java application for building control screens: drag widgets, bind to PVs, save as `.bob XML`. For control screens it is better than anything in the Python design, where the equivalent is a hand-written FastAPI page.

But the access model is the opposite of what was specified. `DESIGN.md` §8 requires everything to be local: loopback binds, control reachable only from the lab PC. Channel Access is designed for the opposite — a broadcast-discoverable network namespace where any machine can `caput`. Restricting it means:

- binding the IOCs to the loopback interface (`EPICS_CAS_INTF_ADDR_LIST=127.0.0.1`), which removes the main advantage of the namespace; or
- using Channel Access security (`ASG` rules in an access file) to restrict write access by host and user.

The second is the proper answer and EPICS supports it well, but it is another configuration file to get right, and a mistake in it is a silent one. In the Python design, "local only" is a bind address.

12. Deployment on Windows

This is the practical obstacle, and it should be evaluated before anything else.

EPICS base	Builds on Windows with Visual Studio, or available prebuilt (conda-forge). Workable.
asyn, StreamDevice, autosave, calc	Must be built against that base. On Linux this is routine; on Windows it is where the time goes, and where a single-maintainer project tends to stall.
Python soft IOC	<code>pcaspy</code> and <code>p4p</code> work on Windows. Fine.
Phoebus	Java, cross-platform. Fine.
Archiver Appliance	Java + Tomcat + MySQL on Windows. Avoid; use the Postgres collector instead.
Running as services	<code>procServ</code> is the Unix convention; on Windows, NSSM wraps each IOC, as in the Python design.

Recommendation if EPICS is chosen: run it on Linux. But the cDAQ requires NI-DAQmx, whose Linux support for this discontinued chassis is exactly the risk that drove the Windows decision in the first place (`XAMS-slow-control-options.pdf` §7). The result is a split system — cDAQ IOC on Windows, everything else on Linux — which is more machines, more to maintain, and harder to debug at two in the morning.

13. Honest comparison

Where EPICS is better:

- Alarm limits, engineering units, deadbands and precision are record fields, not code. This also eliminates the threshold-duplication problem.
- `INVALID` severity on a failed read is automatic — stale data cannot look healthy.
- StreamDevice makes the CAEN and Lake Shore protocols declarative, and roughly half the code of the Python drivers disappears.

- `DRVH` / `DRVL` enforce write limits at the record level.
- `autosave` makes the integrator's persistence a one-line declaration.
- Simulation mode (`SIML` / `SIOL`) is built in, matching `DESIGN.md` §13 without writing it.
- Genuine process isolation per IOC.
- Skills and conventions transfer to and from larger experiments.

Where it is worse, for this system:

- The cDAQ — twenty of the thirty channels — is a Python soft IOC either way.
- Notification (SMS, email) is Python either way, unless Kafka is deployed for one rack.
- Device identification by USB hardware ID and instrument serial, protecting against the two identical CAEN units being confused, must be built as custom logic in an environment less suited to it.
- The flow integrator loses gap accounting and reset-as-period-closure.
- "Local only" runs against the grain of Channel Access.
- Windows deployment is the real obstacle; Linux deployment reintroduces the cDAQ compatibility risk.
- The learning curve is real, and the system is maintained by one person.

The decisive point is unchanged. A one-person EPICS installation nobody else can boot or debug is no more maintainable than the present LabVIEW project. EPICS is worth its cost when a controls group carries it. The question is not whether EPICS is good — it plainly is — but whether XAMS has the surroundings that make it pay.

14. Effort

Assuming the configuration of §3–§8 is already known, as it now is.

Component	Estimate
EPICS base + asyn + StreamDevice + autosave built and working on the target OS	2–5 days , dominated by Windows; the main risk
cDAQ Python soft IOC (pcaspy)	3–4 days
HV IOC — proto, records, substitutions	2 days
Cryo IOC	1 day
UPS IOC	0.5 day
Identity verification gating on the HV writes	1–2 days
CA→PostgreSQL collector + Grafana	2 days
Alarm client → SMS and email	1–2 days
Phoebus screens	2–3 days
Flow integrator with gap accounting and period closure	1–2 days
Commissioning, empirical channel verification, documentation	as in <code>DESIGN.md</code>

Roughly **1.5 to 2 times** the Python design, with the variance concentrated in the first row. If the toolchain builds on the first afternoon, the gap narrows considerably; if it does not, it widens without limit.

15. Open items

Everything in `DESIGN.md` §16 applies unchanged — alarm thresholds, sensor locations, pressure units, HV channel mapping, UPS model, the heater shut-off decision, and the second maintainer.

Additional to this design:

Item	Needed for
Does asyn + StreamDevice build on the target Windows machine?	everything; evaluate first
Which EPICS base version, and who rebuilds it on upgrade	maintenance
Channel Access security file, or loopback binding	§11
Whether Phoebus replaces the web UI entirely, or both exist	§11
Who, besides the primary author, can boot and debug an IOC	the decision itself

16. Recommendation

For XAMS as it stands — four instruments, thirty connected channels, Windows, one maintainer — the Python design in `DESIGN.md` remains the recommendation. The reasons are not architectural: the two designs have the same shape, because EPICS found that shape thirty years ago and the Python design follows it.

The reasons are practical. The largest device needs Python either way, the notification layer needs Python either way, the deployment platform fights EPICS, and there is currently one person to maintain the result.

Reconsider this document if any of the following changes: a second maintainer with controls interest appears; Nikhef controls-group support becomes available; XAMS grows substantially or must integrate with a larger controls effort; or the cDAQ is replaced by hardware with maintained EPICS support.